

FIR Filter Design by Windowing Methods and by PM Algorithm

FIR (Finite Impulse Response) filters have a number of advantages and disadvantages over its counterpart, IIR (Infinite Impulse Response). Due to their finite length, and ability to be made causal, FIR filters have an exactly linear phase, are always stable, and can be designed using linear methods. Their disadvantages, however, are that they typically need a higher filter order to achieve the same level of performance as an IIR filter. A larger delay also goes hand-in-hand with the higher order filter.

In this lab, we need to design a low pass filter using two design methods: first the windowing method, and then the Parks-McClellan method. For the windowing method, we will use the Hanning, Hamming and Blackman windows to create the filter. The low pass filter must satisfy the following specifications:

$$\begin{aligned} 0.95 < H(e^{j\omega}) < 1.05, & \quad 0 \leq |\omega| \leq 0.25\pi, \\ -0.1 < H(e^{j\omega}) < 0.1, & \quad 0.35\pi \leq |\omega| \leq \pi, \end{aligned}$$

Windowing Method

To begin designing a filter using the windowing method, we first need our ideal low pass filter. This will have a cutoff frequency where, at that frequency, a sharp attenuation of the signal occurs thereby blocking any high frequencies. Discontinuities, however, result in a necessarily infinite Fourier Transform, which then requires a non-causal and non-finite-power system. To work around this, we truncate the filter's frequency response using one of our windows, therefore making the system finite. We can also make the system causal by adding a delay of $M/2$ samples, where M is the width of the window. Once we have the ideal impulse response of the filter, we can then multiply by the impulse response of our desired window, and we now have our FIR filter. The work for finding our desired impulse response can be seen below:

Colmberg

Low Pass Filter

• Because it is Causal, we need a delay, so the ideal LPF will be: delay of $\frac{M}{2}$ samples...

$$H_d(e^{j\omega}) = \begin{cases} e^{-j\omega \frac{M}{2}} & 0 \leq \omega \leq \omega_c \\ 0 & \omega_c < |\omega| \leq \pi \end{cases}$$

$$\therefore h_d[n] = \frac{1}{2\pi} \int_{-\omega_c}^{\omega_c} e^{-j\omega \frac{M}{2}} e^{j\omega n} d\omega = \frac{\sin(\omega_c(n - \frac{M}{2}))}{\pi(n - \frac{M}{2})}$$

$$\omega_c = \frac{0.25\pi + 0.35\pi}{2}, \quad h_d[n] = \frac{\sin[(0.3\pi)(n - \frac{M}{2})]}{\pi(n - \frac{M}{2})}$$

Implementing in MATLAB

We are starting with the discrete time system given by

$$h[n] = \frac{\sin[\omega_c(n - M/2)]}{\pi(n - M/2)} w[n],$$

In order to plot the filter, we need to first convert this to the frequency domain. We can do so using the *System Identification Toolbox*, which has the **fft()** function. This function converts a time domain iddata object to a frequency domain iddata object. M , as described by the book, is the order of the system function polynomial for the filter.

Let's start by determining the order of the system. In table 7.2, which we can see below, we note the third column which provides a formula for the approximate width of the main lobe. Since we have a low pass filter with a cutoff normalized discrete frequency of 0.3π , we can determine M . For a **Hann window**, we can do the calculation:

$$\frac{8\pi}{M} = 0.30\pi$$

$$\frac{8\pi}{0.30\pi} = M \approx 26.66$$

Hamming Window

$$\frac{8\pi}{0.30\pi} = M \approx 26.66$$

Blackman Window

$$\frac{12\pi}{0.30\pi} = M = 40$$

TABLE 7.2 COMPARISON OF COMMONLY USED WINDOWS

Type of Window	Peak Side-Lobe Amplitude (Relative)	Approximate Width of Main Lobe	Peak Approximation Error, $20 \log_{10} \delta$ (dB)	Equivalent Kaiser Window, β	Transition Width of Equivalent Kaiser Window
Rectangular	-13	$4\pi/(M+1)$	-21	0	$1.81\pi/M$
Bartlett	-25	$8\pi/M$	-25	1.33	$2.37\pi/M$
Hann	-31	$8\pi/M$	-44	3.86	$5.01\pi/M$
Hamming	-41	$8\pi/M$	-53	4.86	$6.27\pi/M$
Blackman	-57	$12\pi/M$	-74	7.04	$9.19\pi/M$

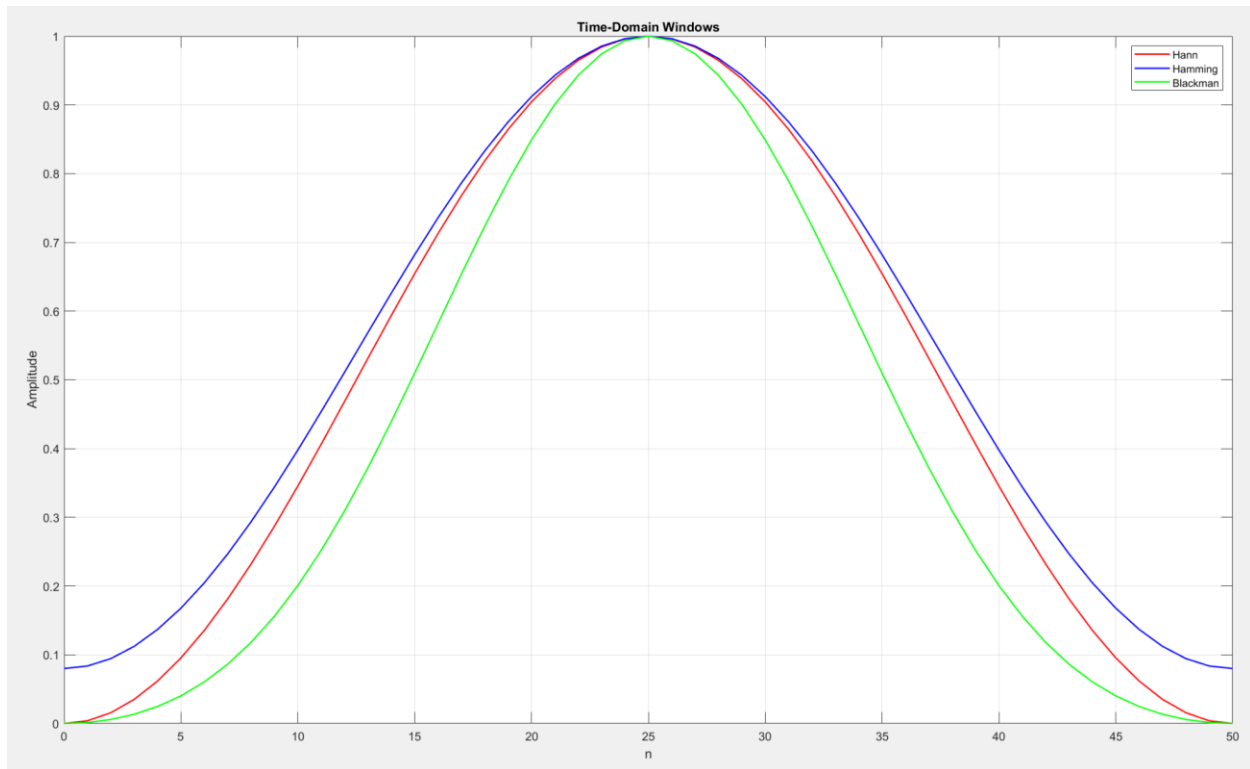
Besides M , I also defined our cutoff frequency as 0.3π , and a sampling frequency of 1kHz. This gives us an analog cutoff frequency of:

$$\begin{aligned}\Omega_c &= \omega_c/T_s = \omega_c F_s \\ \Omega_c &= (0.3\pi)/(2\pi) * 1000 \\ \Omega_c &= 150 \text{ Hz.}\end{aligned}$$

When making the window functions and the windowed impulse responses, I only used $M+1$ data points because all other values would be zero, regardless, due to the truncating effect of the window functions. When transforming to the frequency domain, however, I increased the resolution of our points to 2048 so our frequency response graphs could be more accurate. Because our max frequency is the sampling frequency (in this case 1kHz), the frequency domain will be in increments of $F_s / 2048$, from 0 to F_s .

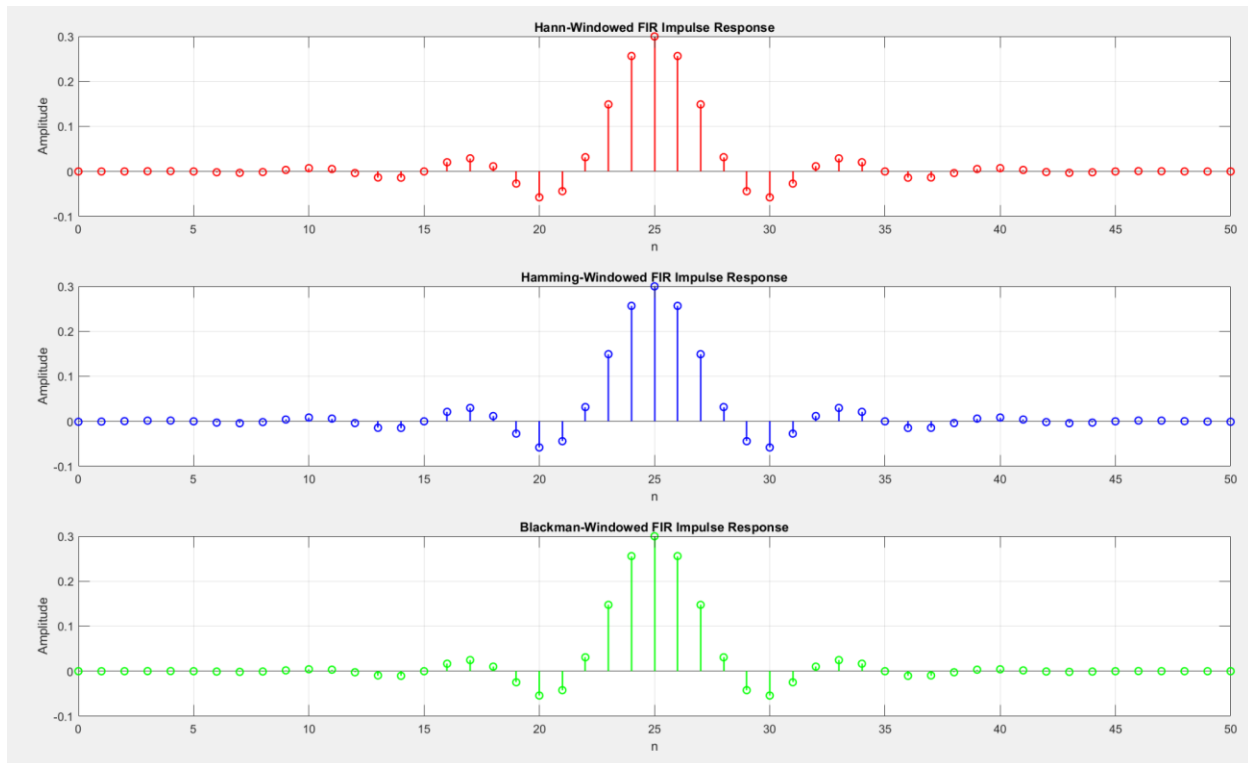
Initial Results

After creating and running the matlab script, these are the results. First, we can plot the windowed functions in the time domain to ensure they are what we want:

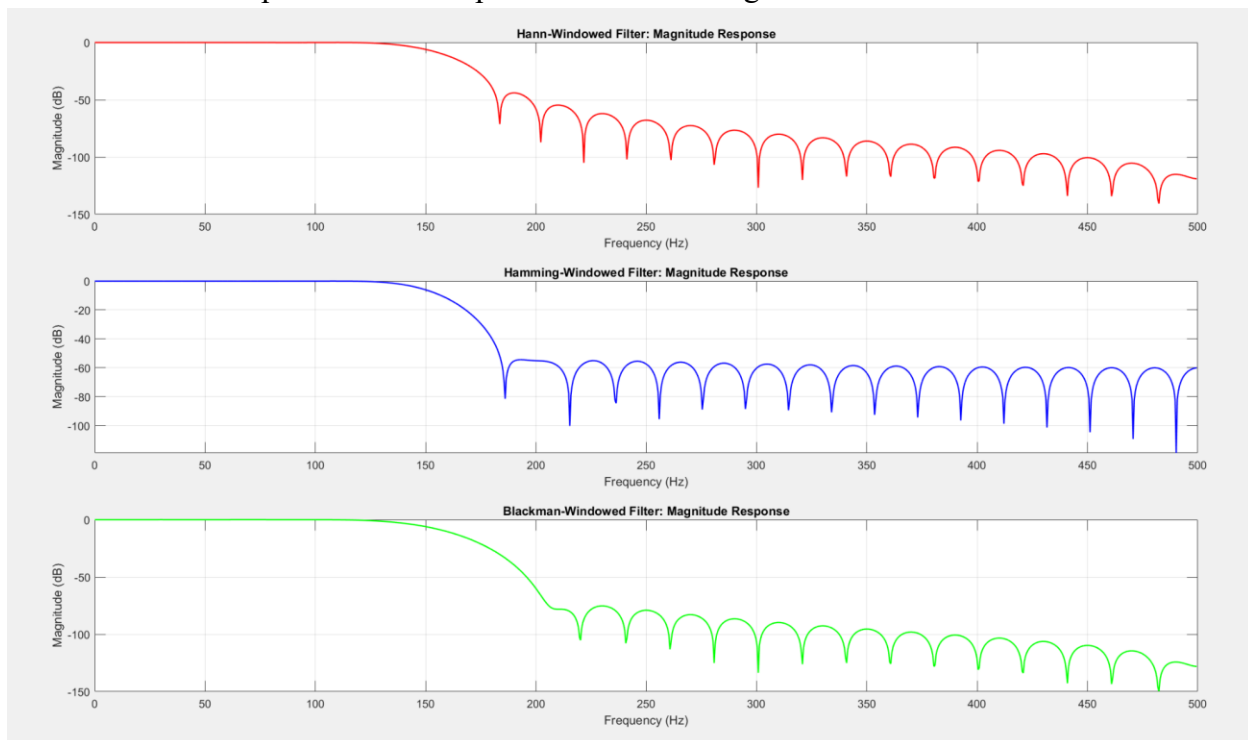


As we can see, the windowed functions match exactly with the graphs seen from the textbook.

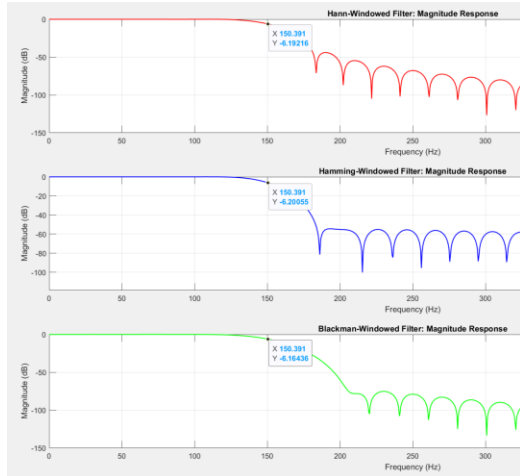
Next, we should look at the windowed impulse responses of the low pass filter. Doing so we can see that they follow a modulated $\text{sinc}()$ function. This is to be expected, given our windowed impulse response is a $\text{sinc}()$ function multiplied by a shifted cosine function.



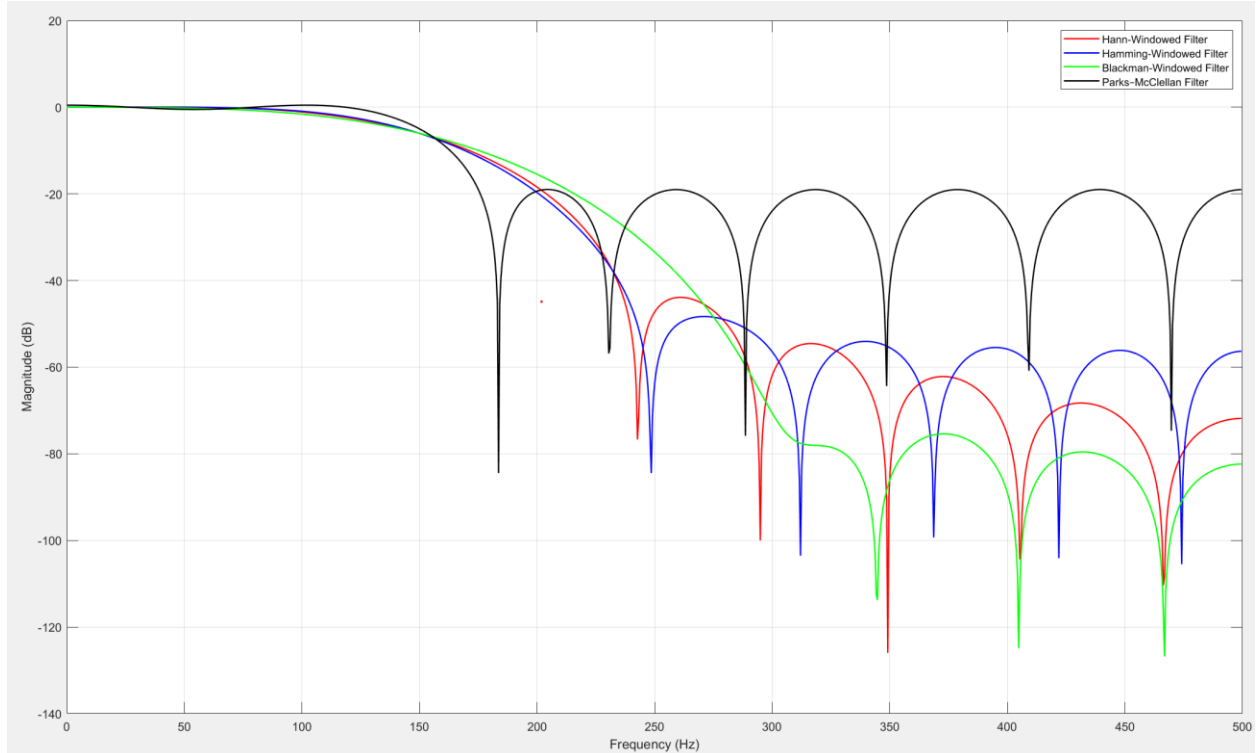
Moving on, I graphed the magnitude and phase response of the FIR filters. We can see that these follow the same shapes we would expect based on the diagrams from the textbook.

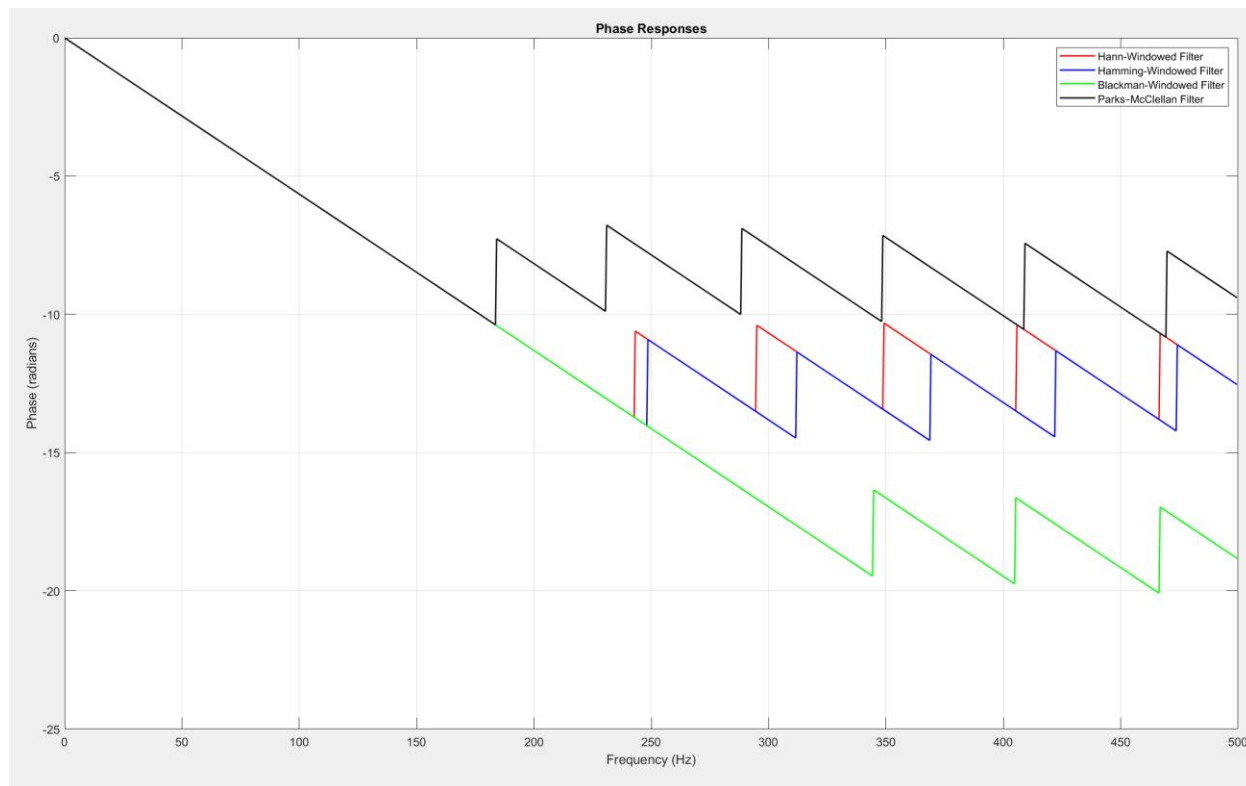


Specifically, we can see that as the width of the main lobe is increased, the height of the side lobes decreases, and vice versa. In the diagram below, we can also see that at our analog cutoff frequency for $F_s = 1\text{kHz}$, each filter is below the -3dB cutoff. Therefore, each filter is sufficient at an order of 50.

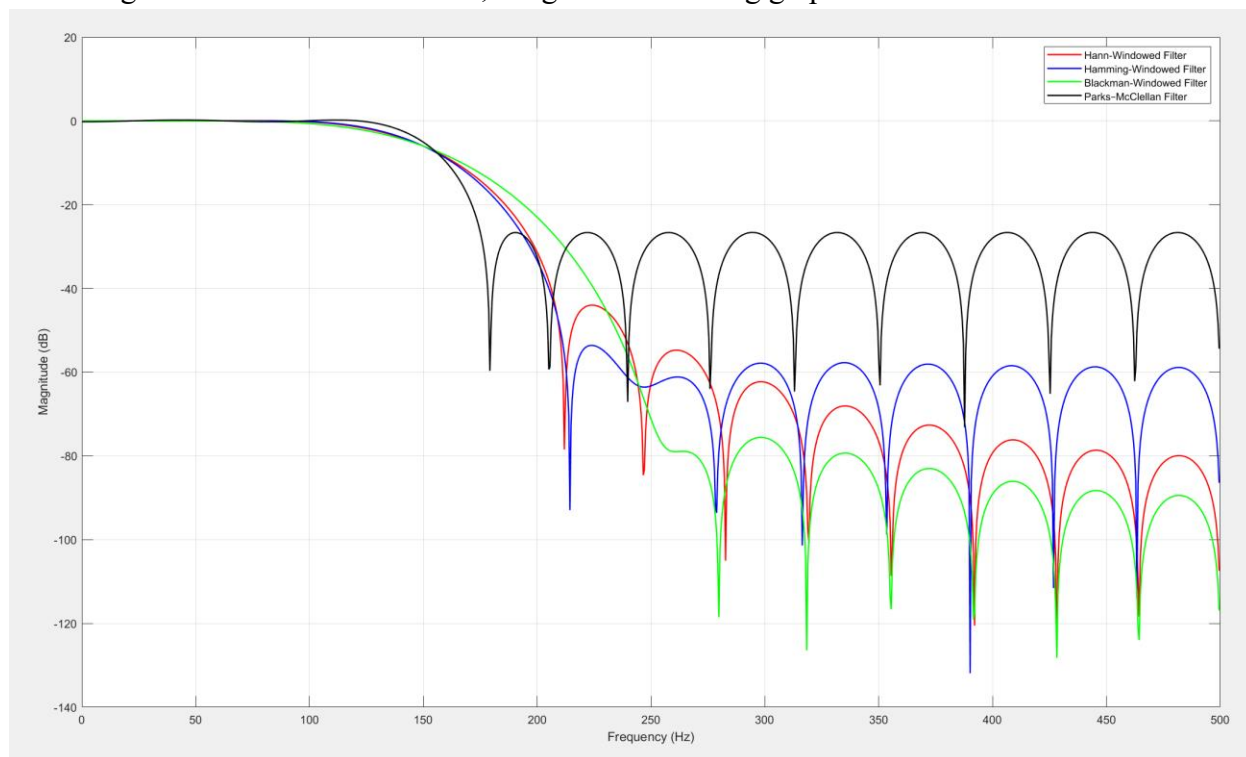


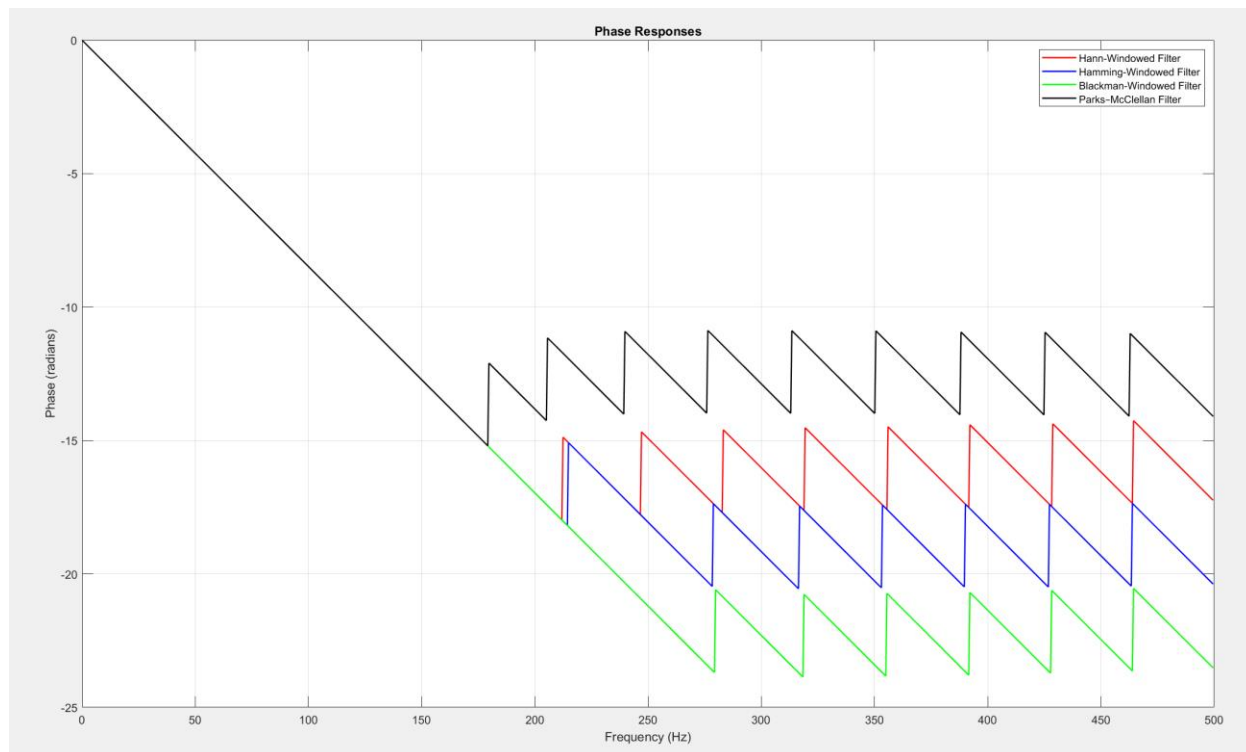
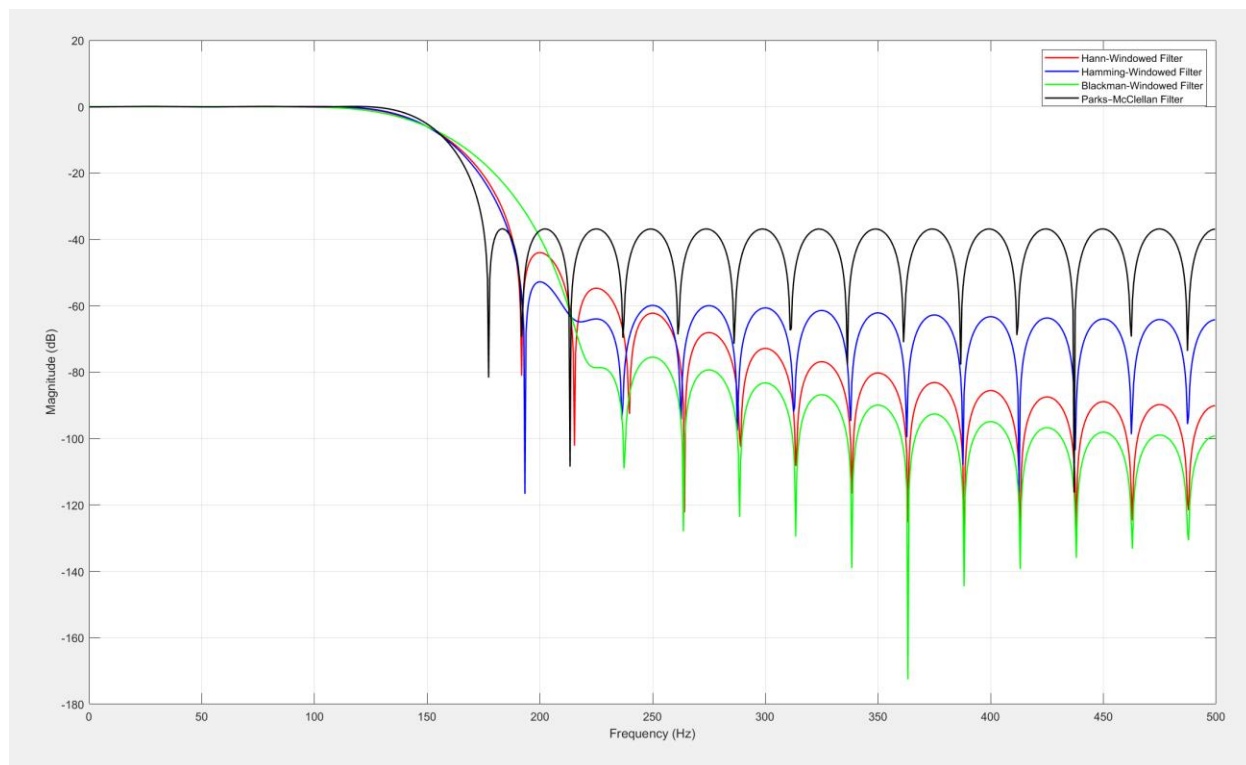
Combining all the graphs and including the Parks-McClellan, which we haven't talked about yet, we get the graph below. For this, the filter order of the window functions and the PM filter is 18.

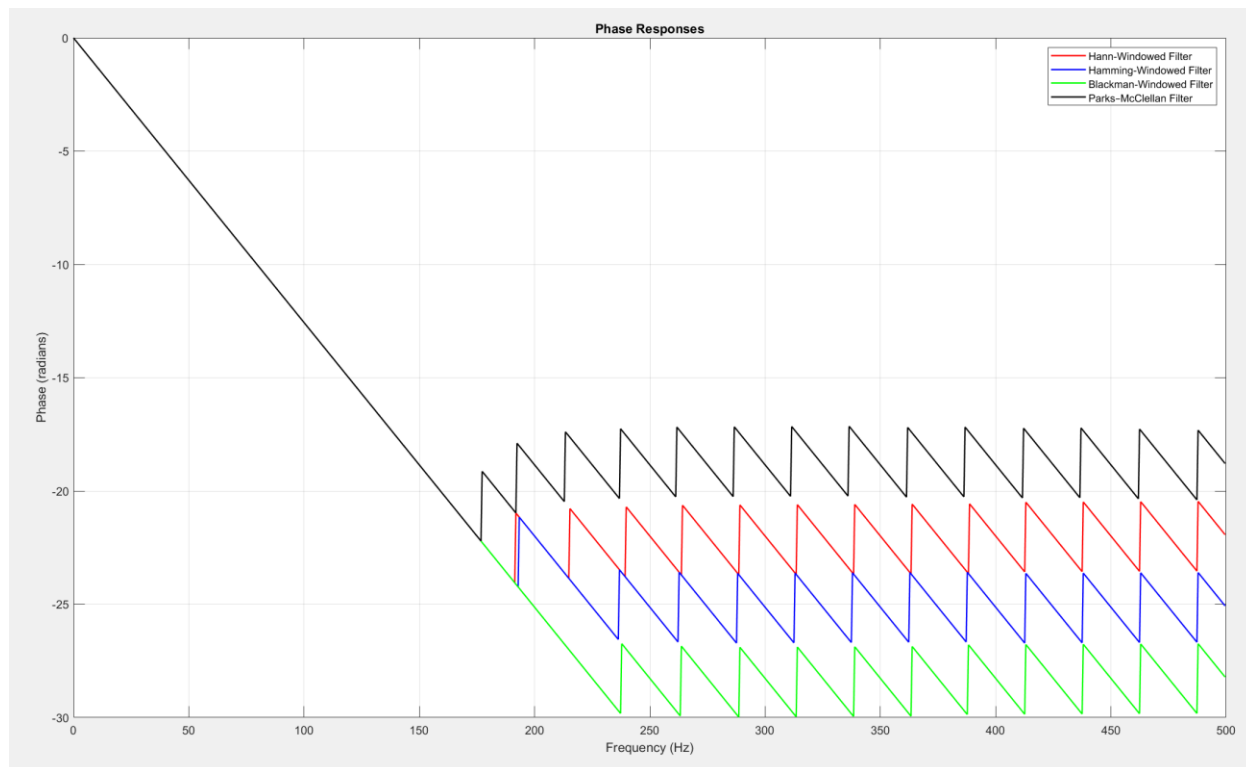
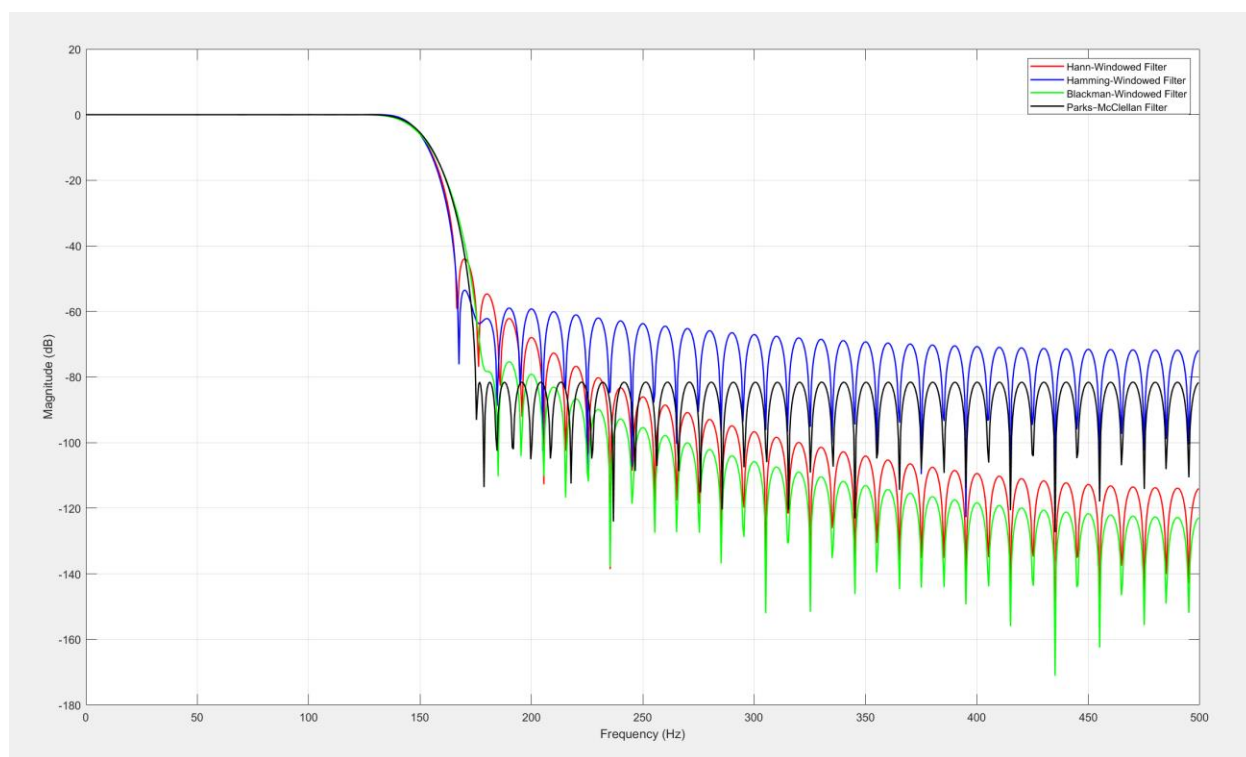


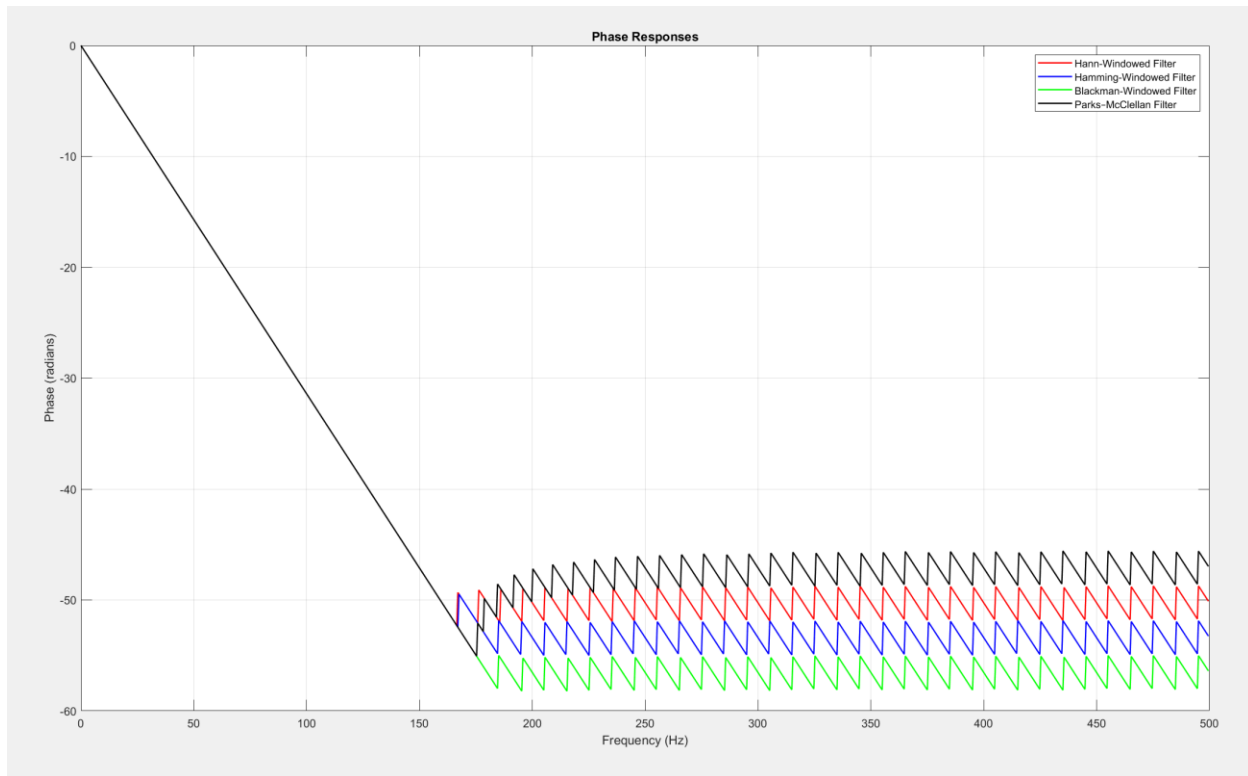


Increasing the filter order to **27**, which should satisfy the approximation for the Hann and Hamming windows discussed earlier, we get the following graph:



**Filter Order 40:**

**Filter Order 100:**



Observing the graphs above, there are many characteristics to observe about the filters as the filter order increases. Firstly, we can notice that the magnitude of both the filter attenuation and phase negatively increase as the filter order increases. This relationship appears to be non-linear. For example, the PM filter has a highest equi-ripple attenuation in the stop-band of -20dB, then -30 dB at a filter order of 27, and -35 dB at $M=40$. With a filter order of 100, the highest value in the stop-band of the PM filter is -85 dB! Along with the increased attenuation for each filter, it can also easily be observed that the transition width and width of side lobes decreases with M . These follow what we were told by table 7.2.

It is interesting to note that in the graph for the filters with an order of 100, we can see that the PM attenuates the signal below all other signals for a window of frequency, and stays below the hamming window entirely. It does have a slightly wider main lobe than the Hann and Hamming window filters, however.

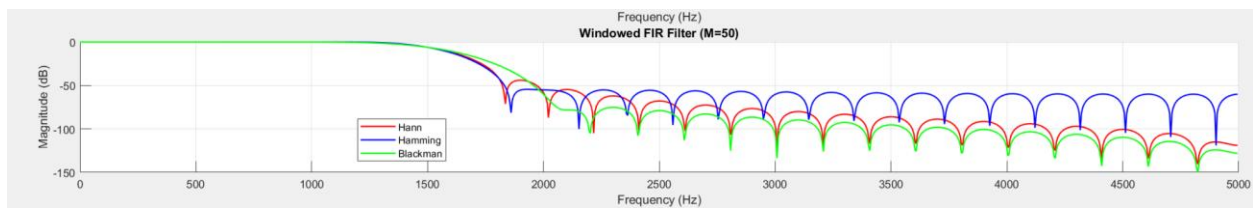
Looking at the phase graphs, notice that the phase increases (negatively) with the filter order. But, the frequency at which it stops being linear and enters the ripple part stays the same. This translates to a growing slope of the linear line, probably in a linear manner with the increasing magnitude of the phase. What this tells us is that the group delay of the system increases with the order of the filter.

Changing the Sampling Frequency, F_s

Based on the equation

$$\Omega_C = \omega_C/T_S = \omega_C F_S,$$

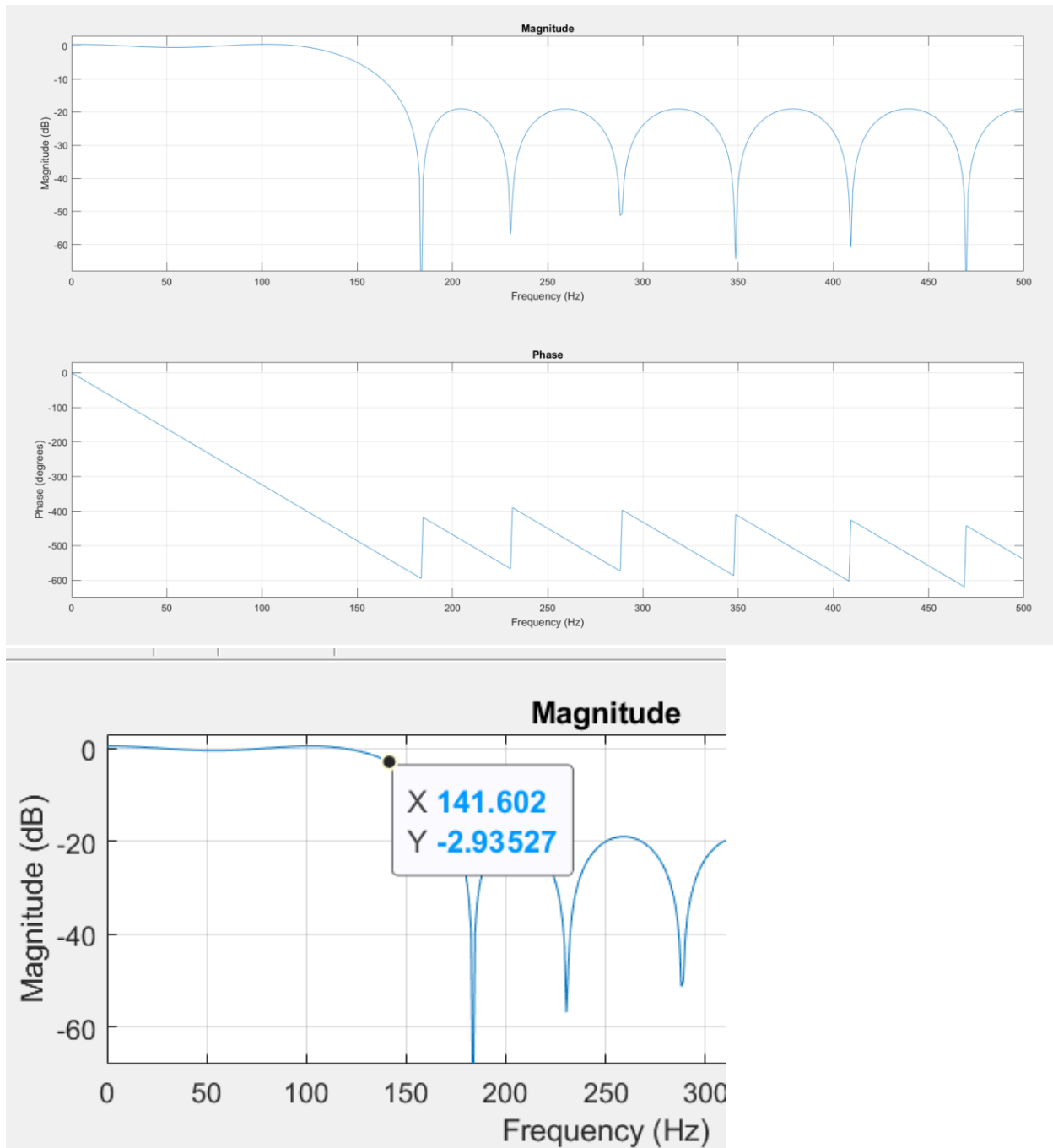
We know that it is only the analog frequency of the filter which will change if we change the sampling frequency. We can see this directly by replotting the magnitude graph, but this time with a sampling frequency of 10kHz. As seen in the image below, the frequency on the x-axis has just been scaled up 10 times, as we expected since F_s was scaled up by 10 times.



Parks-McClellan Method

Implementing the Parks-McClellan method is much easier in matlab than following the algorithm from the textbook. There are two functions we can use which do the whole process for us: those being **firpmord** and **firpm**. First, **firpmord()** works to estimate the order of the filter. It does this by taking in vectors for the pass band and stop bands edges, the sampling frequency, the desired amplitude in each band region, and the allowable rippled deviation from the desired amplitude in each region. With these inputs, it then returns the estimated order, an amplitude vector, the band edges, and weights. With these outputs, we can then plug them directly into **firpm()** to get the filter that minimizes the maximum error from the ideal filter. It also aims to get equal error throughout in each band which is where the equi-ripple behavior originates from.

After running my script, we can see the results below. The calculated filter order was 18, and it had a -3dB value at around 140Hz, which is an improvement from the windowing method filters which had a similar -3dB value with an order of 50.



Comparing the Windowing Method to Parks-McClellan Algorithm

The windowing method focuses on the fact that the ideally desired filters are not realizable as they are non-causal and have an infinite energy spectrum. By truncating the frequency response of this filter, however, and adding a delay, we can make the system function for the filter be both

causal and finite. Because we want our impulse response to be finite and causal, we multiply the time-domain impulse response by our window. This leads to a convolution in the frequency domain between our ideal filter and the fourier transform of the window. This convolution yields our final filter that we will implement. We can adjust parameters like the sampling and cutoff frequency, which will determine the analog cutoff frequency, and the order of the filter, which can also be represented as the number of samples long the window is.

On the other hand, the Parks-McClellan algorithm is a process that strives to find the “optimal” filter for a given order. In order to define this optimum filter, we define the error between our designed filter and the ideal filter. We then minimize the maximum error in the pass and stop bands. This is the function the Parks-McClellan algorithm performs. To do this, the algorithm successively approximates the weighted coefficients of a polynomial series of cosines of a specified frequency. It is very laborious to do it ourselves, but the `firpm()` matlab function performs this algorithm for us. A side-effect of the Parks-McClellan algorithm is that the filter will have the same deviation in both the stop-band and the pass-band, and the maximum deviation will be the same at a specific number of points, specified by the order of the filter. This creates a phenomenon known as ‘equi-ripple’, since we have equally high, periodic, values which create ripples in the stop-band (since this is a low pass filter).

From the results above, and the pictures below, we can see that the Parks-McClellan method created a filter that, at a specific filter order, had a significantly greater attenuation after the cut-off frequency. It did have higher deviation in the ripples of the stop-band, however, for a filter order of 18, it performed better than a filter of order 50 using the windowing method.

